

-  Project Tasks and Instructions
 -  Overview
 -  Task 1: Create Architecture and Dataflow Diagrams using Mermaid
 - Objective
 - Requirements
 - Deliverables
 - Format Requirements
 - Example Structure
 - Component Details
 -  Task 2: Implement Logging Functionalities
 - Objective
 - Requirements
 - Implementation Guidelines
 - Deliverables
 - Example Log Format
 - Reference
 -  Task 3: TBD
 -  Submission Checklist
 - Task 1
 - Task 2
 -  Additional Resources
 -  Notes
 -  Notes

Project Tasks and Instructions

This document outlines the tasks to be completed for the Speech-to-Text with Speaker Diarization project, based on the [Interview Preparation Instructions](#).

Overview

This project implements a speech-to-text transcription system with speaker diarization capabilities. The following tasks will enhance the codebase with proper documentation and logging functionality.



Task 1: Create Architecture and Dataflow Diagrams using Mermaid

Objective

Create comprehensive visual documentation of the system architecture and data flow using Mermaid diagrams.

Requirements

1. High-Level Architecture Diagram

- Create a diagram showing the overall system architecture
- Include all major components:
 - Audio Input Processing
 - Speaker Diarization Pipeline (pyannote.audio)
 - Speech Transcription (Whisper)
 - Segment Processing and Merging
 - Output Generation
- Show relationships and data flow between components
- Use Mermaid syntax (flowchart, graph, or architecture diagram)

2. Dataflow Diagram (Component-Level)

- Create a detailed dataflow diagram showing:
 - **Inputs received by each component**
 - **Processing performed within each component**
 - **Outputs generated by each component**
- Include all functions:
 - `segment_to_float32()`
 - `merge_same_speaker()`
 - `split_overlaps()`
 - `build_clean_segments()`
- Show data transformations at each step
- Include external dependencies (Hugging Face models, Whisper models)

Deliverables

Create two separate Markdown files:

1. **architecture.md** - High-level architecture diagram
2. **dataflow.md** - Component-level dataflow diagram

Format Requirements

- Use Mermaid syntax for all diagrams
- Diagrams should be embedded in Markdown code blocks with **mermaid** language identifier
- Include descriptive text explaining each component
- Reference the sample projects mentioned in [interview-prep-instructions.md](#):
 - [Face Recognition Project - Architecture](#)
 - [Face Recognition Project - Component Diagram](#)
 - [Gesture Recognition - Project Overview](#)
 - [Gesture Recognition - Component Diagram](#)

Example Structure

```
# Architecture Diagram
```

```
## System Overview
```

```
[Description of the overall system]
```

```
```mermaid
```

```
[Your Mermaid diagram code here]
```

## Component Details



## Task 2: Implement Logging Functionalities

# Objective

Enhance the codebase with comprehensive logging using Python's `logging` module, following the requirements specified in [interview-prep-instructions.md](#).

## Requirements

Based on the **Logging Requirements** section in the interview prep instructions, implement logging that includes:

### 1. Metadata about inputs and outputs of key functions

- Log function entry with input parameters
- Log function exit with output data/metadata
- Include data types and shapes where applicable

### 2. Object data length and significant object changes

- Log array/segment lengths
- Log number of segments before/after processing
- Track data transformations

### 3. Execution time and space/memory usage

- Measure and log execution time for major processes:
  - Audio loading and preprocessing
  - Speaker diarization pipeline execution
  - Segment processing (merging, splitting)
  - Whisper transcription (per segment and total)
  - Overall execution time
- Log memory usage at key points
- Use appropriate tools (e.g., `time`, `memory_profiler`, or `psutil`)

### 4. Time and space complexity observations

- Document algorithmic complexity where applicable
- Note complexity for:
  - `merge_same_speaker()` -  $O(n)$  where  $n$  is number of segments
  - `split_overlaps()` -  $O(n)$  where  $n$  is number of segments
  - Overall processing complexity

## 5. Exceptions, errors, and unusual behavior

- Wrap critical sections in try-except blocks
- Log all exceptions with full stack traces
- Log warnings for unusual conditions (e.g., empty segments, failed transcriptions)
- Log errors for missing files, invalid tokens, etc.

# Implementation Guidelines

## 1. Create a dedicated logger configuration

- Set up logging in a separate module or at the top of `app.py`
- Use appropriate log levels (DEBUG, INFO, WARNING, ERROR, CRITICAL)
- Configure both console and file handlers
- Use formatted log messages with timestamps

## 2. Log file naming

- Use timestamped log files (e.g., `logs/app_YYYY-MM-DD_HH-MM-SS.log`)
- Create a `logs/` directory if it doesn't exist

## 3. Key functions to instrument

- `segment_to_float32()` - Log input/output shapes, data types
- `merge_same_speaker()` - Log segment counts, merge operations
- `split_overlaps()` - Log overlap resolution operations
- `build_clean_segments()` - Log filtering and processing steps
- Main execution flow - Log each major step

## 4. Performance tracking

- Use decorators or context managers for timing functions
- Log memory usage before/after major operations
- Track processing time per segment

# Deliverables

1. Enhanced `app.py` with comprehensive logging

2. **Log files** generated during execution (in **logs/** directory)
3. **Documentation** of logging approach in code comments

# Example Log Format

2024-01-15 10:30:45,123 - INFO - Starting audio processing: audio.mp3 2024-01-15  
10:30:45,125 - INFO - Audio loaded: duration=120.5s, channels=1, sample\_rate=16000  
2024-01-15 10:30:45,200 - INFO - Speaker diarization started 2024-01-15 10:31:20,450  
- INFO - Diarization completed: 45 segments found, execution\_time=35.25s 2024-01-15  
10:31:20,451 - INFO - Building clean segments: input=45, after\_filtering=42 2024-01-15  
10:31:20,460 - INFO - Merging segments: before=42, after=38 2024-01-15  
10:31:20,470 - INFO - Splitting overlaps: before=38, after=35 2024-01-15 10:31:20,471  
- INFO - Starting transcription of 35 segments 2024-01-15 10:31:25,123 - INFO -  
Segment [5.23s -- 8.45s] SPEAKER\_00: transcribed, duration=0.5s ... 2024-01-15  
10:35:30,789 - INFO - Transcription completed: 35 segments processed,  
total\_time=245.67s 2024-01-15 10:35:30,790 - INFO - Output written to: transcript.txt

# Reference

See the **Logging Requirements** section in [interview-prep-instructions.md](#) for complete specifications.



## Task 3: TBD

**Status:** This task will be updated after submission of Task 1 and Task 2.



# Submission Checklist

## Task 1

- ☐ **architecture.md** created with high-level architecture diagram
- ☐ **dataflow.md** created with component-level dataflow diagram

- ☐ All diagrams use Mermaid syntax
- ☐ Diagrams are properly embedded in Markdown
- ☐ Component inputs, processing, and outputs are clearly documented

## Task 2

- ☐ Logging module configured with appropriate handlers
  - ☐ All key functions have logging statements
  - ☐ Execution time tracking implemented
  - ☐ Memory usage tracking implemented (where feasible)
  - ☐ Exception handling with logging implemented
  - ☐ Log files are generated in **logs/** directory
  - ☐ Log format includes timestamps and appropriate detail levels
- 



## Additional Resources

---

- [Mermaid Documentation](#)
  - [Python Logging Documentation](#)
  - [Interview Preparation Instructions](#)
  - Sample Projects:
    - [Face Recognition System](#)
    - [Gesture Recognition](#)
- 



## Notes

---

- Ensure all diagrams are clear and follow the structure of the sample projects
- Logging should be comprehensive but not overly verbose
- Test logging with actual audio files to ensure proper functionality
- Document any assumptions or limitations in your implementation
- Sample Projects:

- [Face Recognition System](#)
  - [Gesture Recognition](#)
- 



## Notes

---

- Ensure all diagrams are clear and follow the structure of the sample projects
- Logging should be comprehensive but not overly verbose
- Test logging with actual audio files to ensure proper functionality
- Document any assumptions or limitations in your implementation